interleaved one after the other must be thrown away. Not only do you have to account for things happening truly concurrently, but *threads don't have to agree on the order of events*. In order to write (or even just to understand) any code that uses a memory ordering other than the default `memory_order_seq_cst`, it's absolutely vital to get your head around this. It's not just that the compiler can reorder the instructions. Even if the threads are running the same bit of code, they can disagree on the order of events because of operations in other threads in the absence of explicit ordering constraints, because the different CPU caches and internal buffers can hold different values for the same memory. It's so important I'll say it again: *threads don't have to agree on the order of events*.

Not only do you have to throw out mental models based on interleaving operations, you also have to throw out mental models based on the idea of the compiler or processor reordering the instructions. *In the absence of other ordering constraints, the only requirement is that all threads agree on the modification order of each individual variable*. Operations on distinct variables can appear in different orders on different threads, provided the values seen are consistent with any additional ordering constraints imposed.

This is best demonstrated by stepping completely outside the sequentially consistent world and using `memory_order_relaxed` for all operations. Once you've come to grips with that, you can move back to acquire-release ordering, which allows you to selectively introduce ordering relationships between operations and claw back some of your sanity.

Relaxed Ordering

Operations on atomic types performed with relaxed ordering don't participate in synchronizes-with relationships. Operations on the same variable within a single thread still obey happens-before relationships, but there's almost no requirement on ordering relative to other threads. The only requirement is that accesses to a single atomic variable from the same thread can't be reordered; once a given thread has seen a particular value of an atomic variable, a subsequent read by that thread can't retrieve an earlier value of the variable. Without any additional synchronization, the modification order of each variable is the only thing shared between threads that are using `memory_order_relaxed`.

To demonstrate just how relaxed your relaxed operations can be, you need only two threads, as shown in the following listing.

Listing 5.5. Relaxed operations have very few ordering requirements